

Tools for software development

Lab 5 - Robot Framework and Keyword-Based Testing



Where we are in the course

The project moves from setup to professional test documentation.



Lab 5 goals

Move from Selenium actions to reusable keywords and reports

- Understand the purpose of Robot Framework
- Create and run .robot test suites
- Use variables, settings and SeleniumLibrary
- Refactor repeated steps into custom keywords
- Generate and interpret HTML reports and logs
- Rewrite at least one Selenium scenario from Lab 4

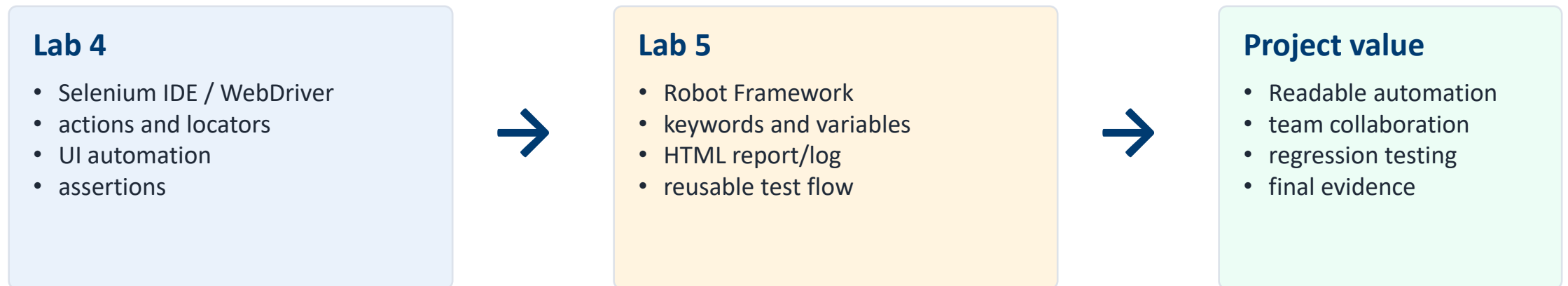
Expected team output

- 2 Robot Framework test cases
- 1 positive test
- 1 negative or validation test
- custom keywords
- HTML report/log
- Git repository update



Lab 4 → Lab 5

Same scenarios, more readable test design



Robot Framework does not remove Selenium. It makes test scenarios easier to read and reuse.



Code-based vs keyword-based testing

Readable steps help testers and developers collaborate

```
driver.findElement(By.id("user-name"))  
    .sendKeys("standard_user");  
driver.findElement(By.id("password"))  
    .sendKeys("secret_sauce");  
driver.findElement(By.id("login-button"))  
    .click();
```

Selenium WebDriver



```
Input Text      id:user-name      standard_user  
Input Text      id:password      secret_sauce  
Click Button    id:login-button  
Page Should Contain  Products
```

Robot Framework



Lab 5 goals

Move from Selenium actions to reusable keywords and reports

- Understand the purpose of Robot Framework
- Create and run .robot test suites
- Use variables, settings and SeleniumLibrary
- Refactor repeated steps into custom keywords
- Generate and interpret HTML reports and logs
- Rewrite at least one Selenium scenario from Lab 4

Expected team output

- 2 Robot Framework test cases
- 1 positive test
- 1 negative or validation test
- custom keywords
- HTML report/log
- Git repository update



Installation

Prepare Python, Robot Framework and SeleniumLibrary

1. Verify Python

```
python --version  
pip --version
```

2. Install Robot

```
pip install robotframework
```

3. Add SeleniumLibrary

```
pip install robotframework-seleniumlibrary
```

```
robot --version
```

If installation fails, use a virtual environment or check PATH settings.



Robot Framework

- Universal tool for test automation
- Control using commands and keywords
- Output (report) in HTML/XML format
- Easy integration into tools
- Freely available
- <https://robotframework.org/>



Installation interpreter

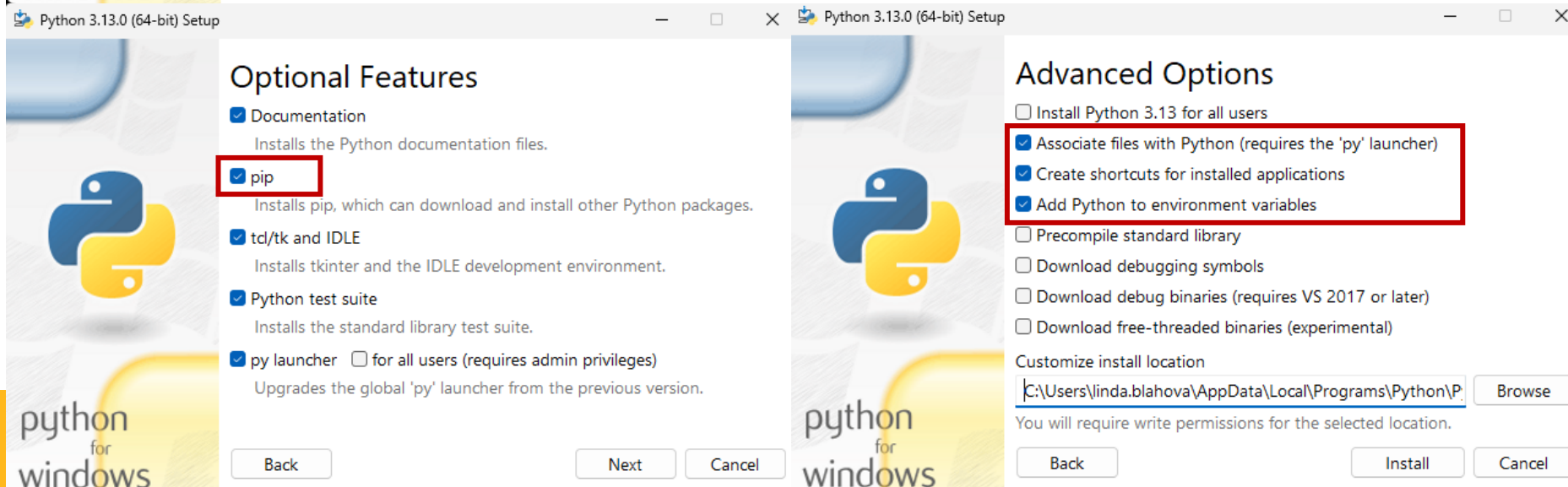
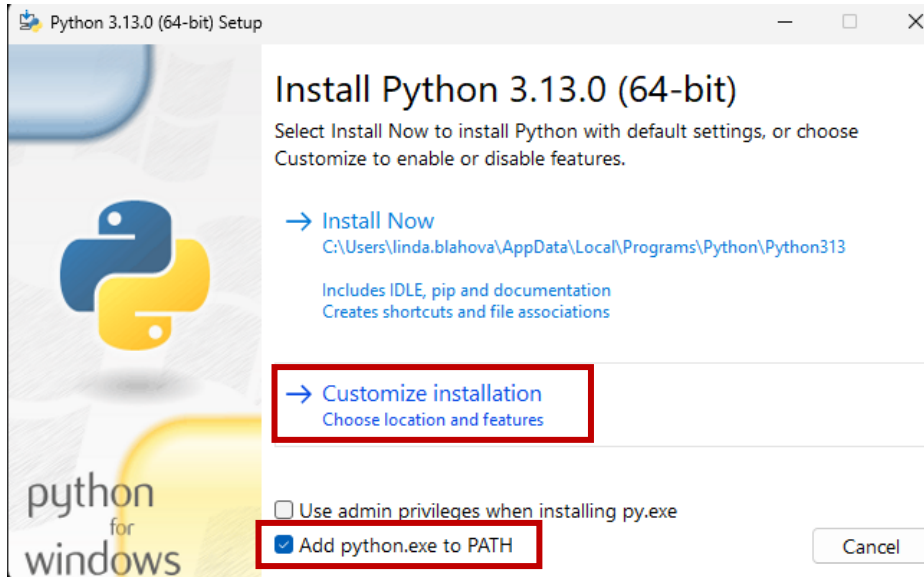
Python , Jython (JVM), IronPython (.NET)

<https://www.python.org/downloads/>



Installation interpreter

- Essential:
 - Adding python.exe to the **PATH environment** variable
 - Installing **pip** (package installer for python)



Verifying Python installation

- Verifying the latest version of python: `python -V`



```
Microsoft Windows [Version 10.0.22621.4460]
(c) Microsoft Corporation. All rights reserved.

C:\Users\linda.blahova>python -V
Python 3.13.0
```

- View all installed versions /since installed – Control Panel

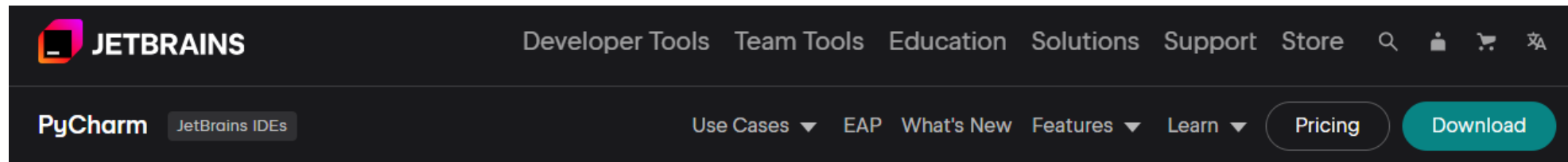
Name	Publisher	Installed On	Size	Version
 Python 3.9.13 (64-bit)	Python Software Foundation	17. 9. 2024	102 MB	3.9.13150.0
 Python 3.10.9 (64-bit)	Python Software Foundation	17. 9. 2024	101 MB	3.10.9150.0
 Python 3.12.6 (64-bit)	Python Software Foundation	17. 9. 2024	151 MB	3.12.6150.0
 Python 3.13.0 (64-bit)	Python Software Foundation	29. 11. 2024	153 MB	3.13.150.0

- Python will not run the command unless the path to `python.exe` is in the PATH variable (option to manually add it)
 - Usually: `C:\Users\[ username ]\AppData\Local\Programs\Python\Python [ version ]\`



Installing/ USE the IDE

<https://www.jetbrains.com/pycharm/download/?section=windows>



Windows macOS Linux

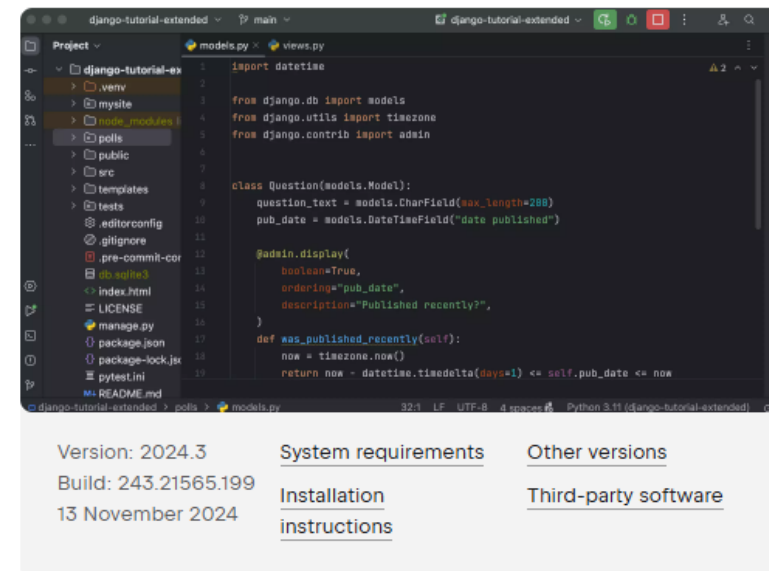
PyCharm Professional

The Python IDE for data science and web development

Download

.exe (Windows) ▼

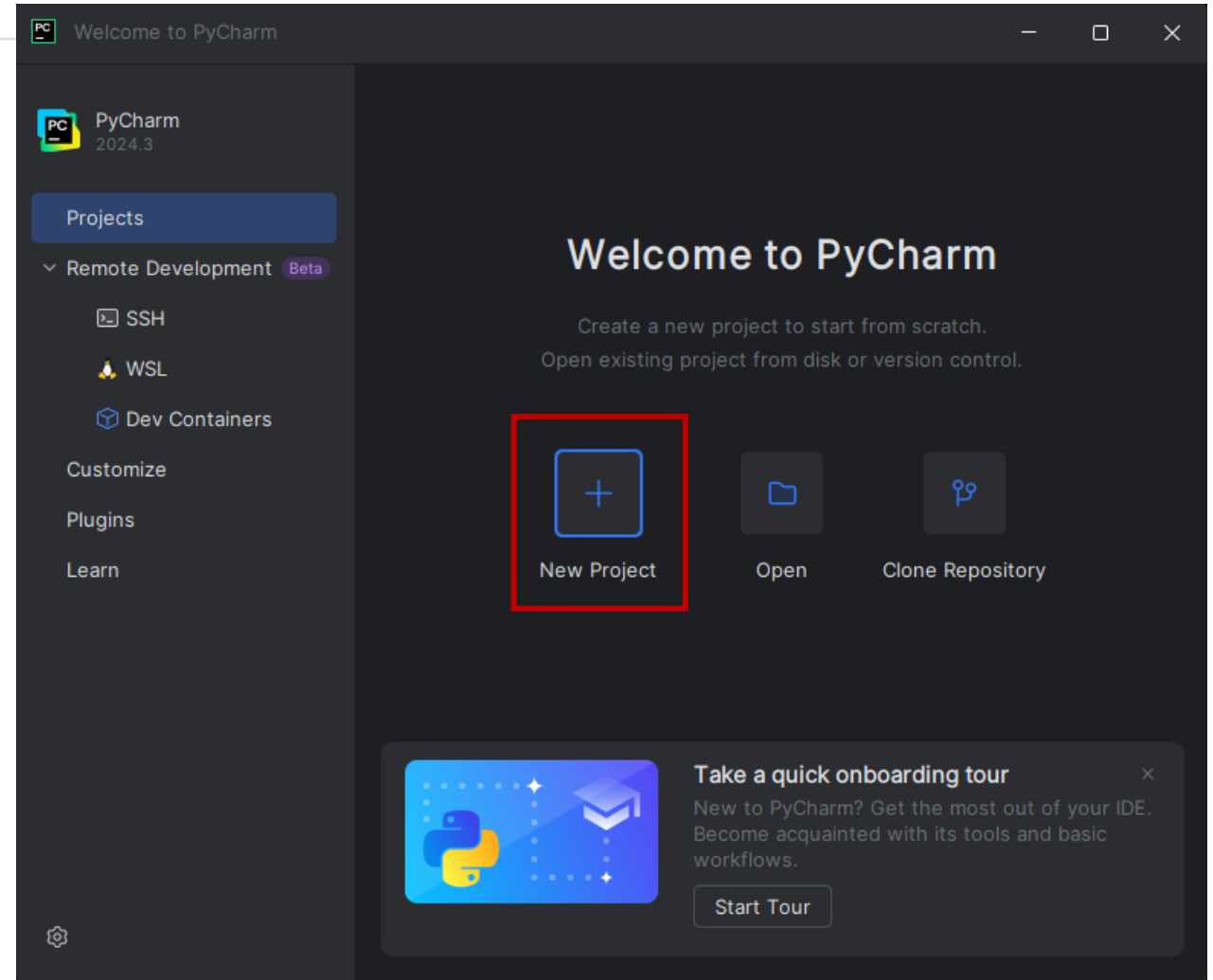
Free 30-day trial



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

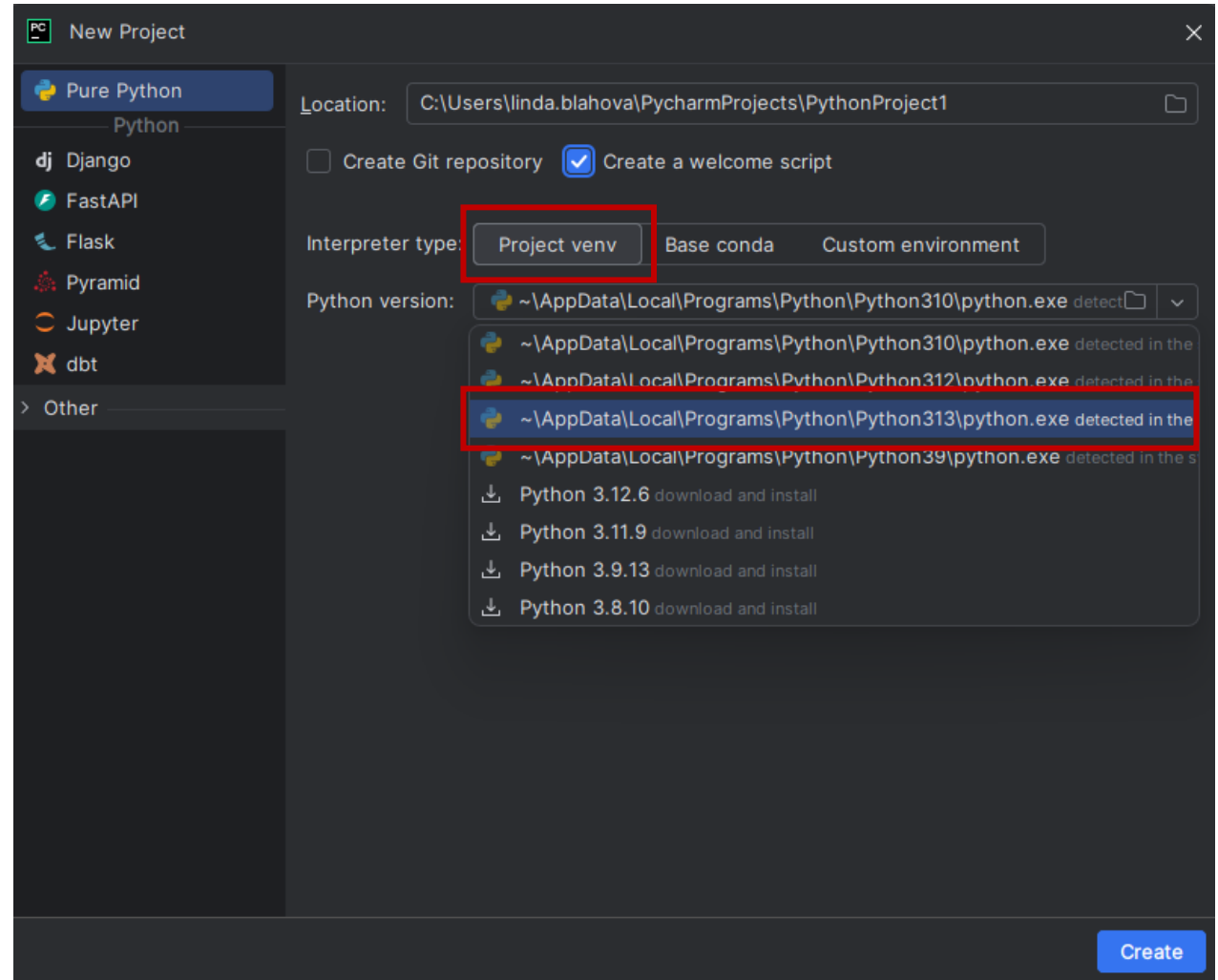
Creating a new project

- New Project
- Other options:
 - Creating a project from an existing folder
 - Clone repository from VC system



Creating a new project – creating a virtual environment

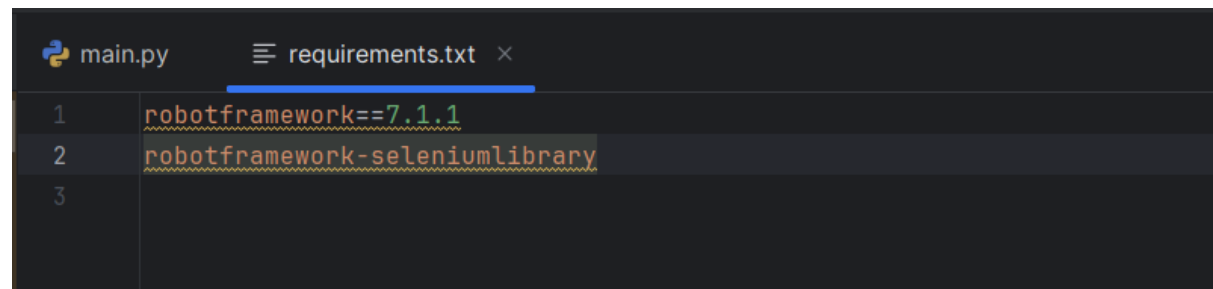
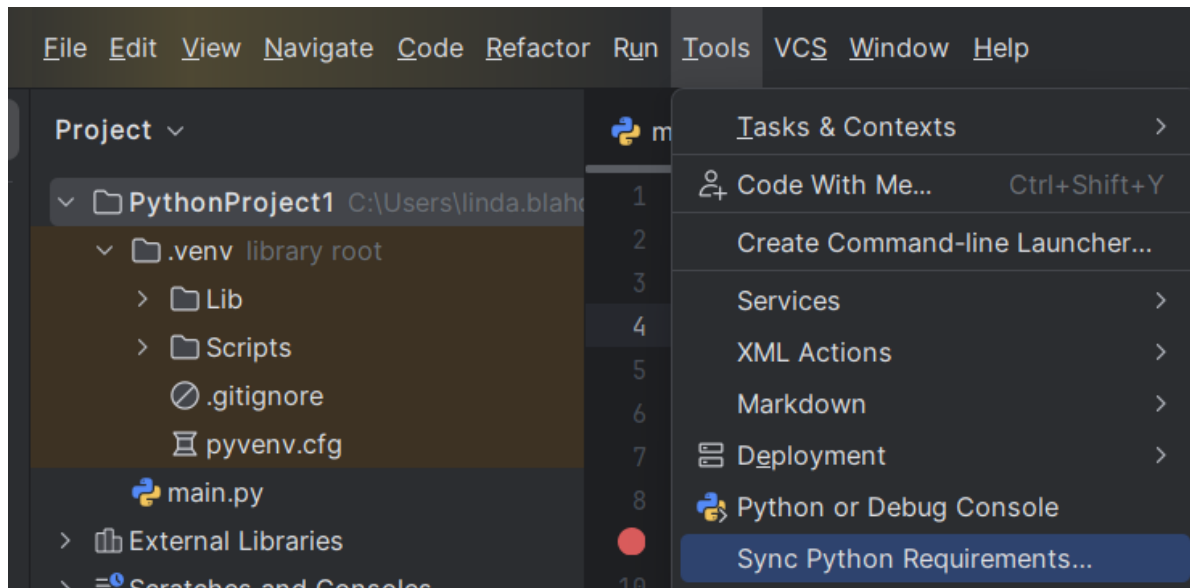
- Virtual environment:
 - Separate for each project
 - Option to choose Python version



Virtual environment configuration

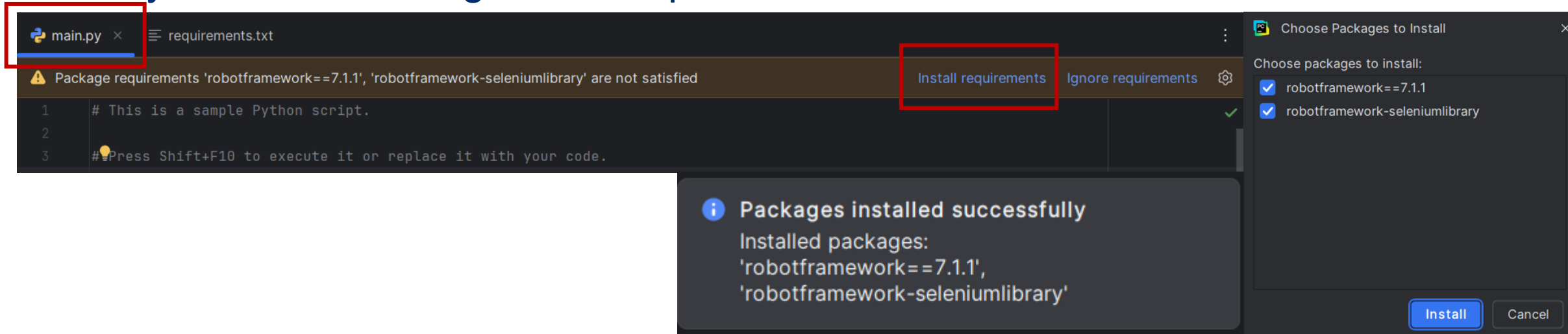
- Tools → Sync Python Requirements
Creation **requirements.txt** with outfit
- Specification of libraries and packages used
in the project
Specific or minimal versions
- Replaces console commands
pip install [packagename]
- Compatibility and easy project set-up

```
robotframework==7.1.1  
robotframework-seleniumlibrary
```



Installing dependencies from requirements.txt

- Automatic detection of installed and missing dependencies
 - It will appear after clicking into the .py file
- Easy installation using Install requirements



- Or via terminal:

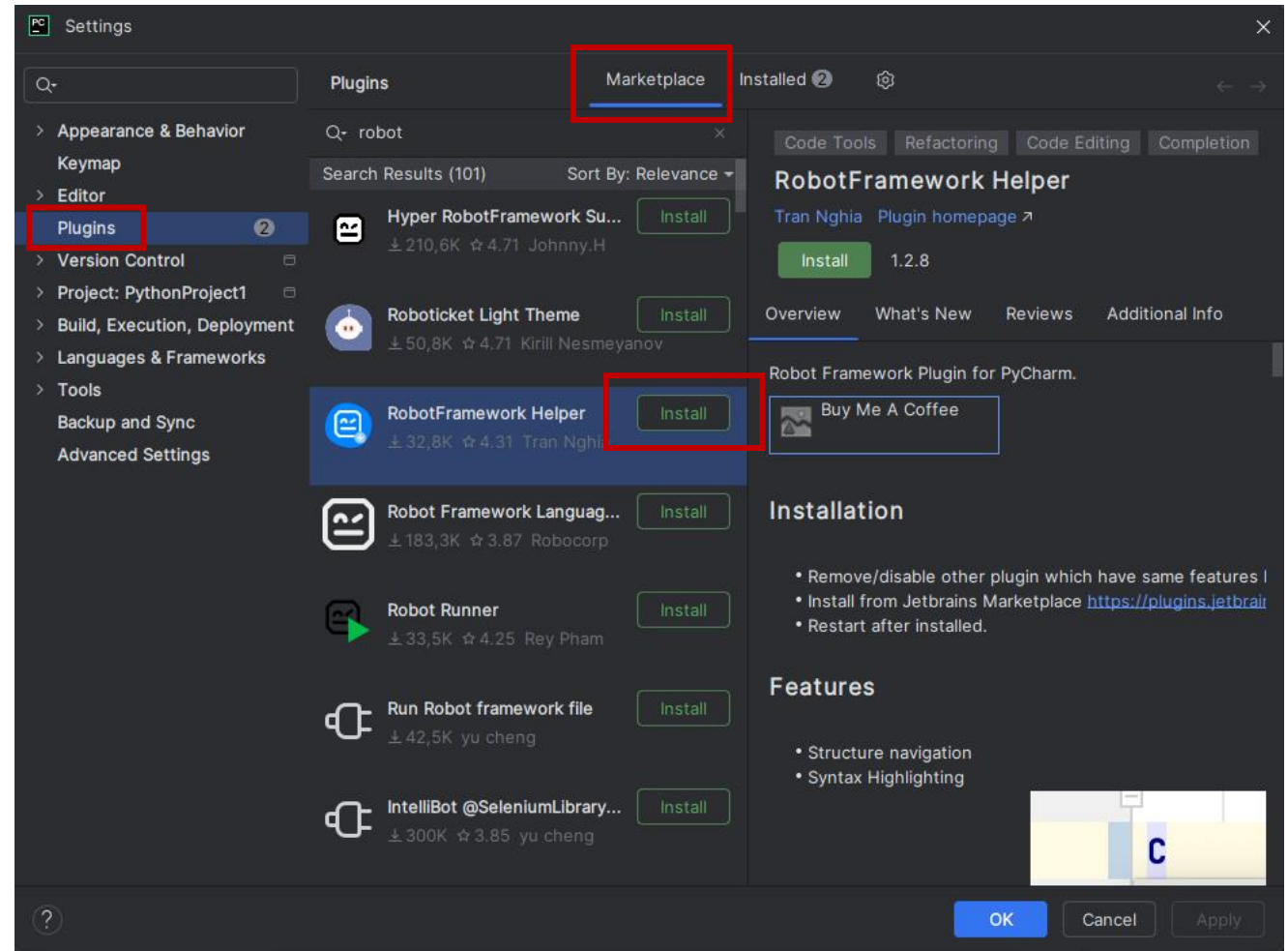
Python terminal

```
pip install -r requirements.txt
```



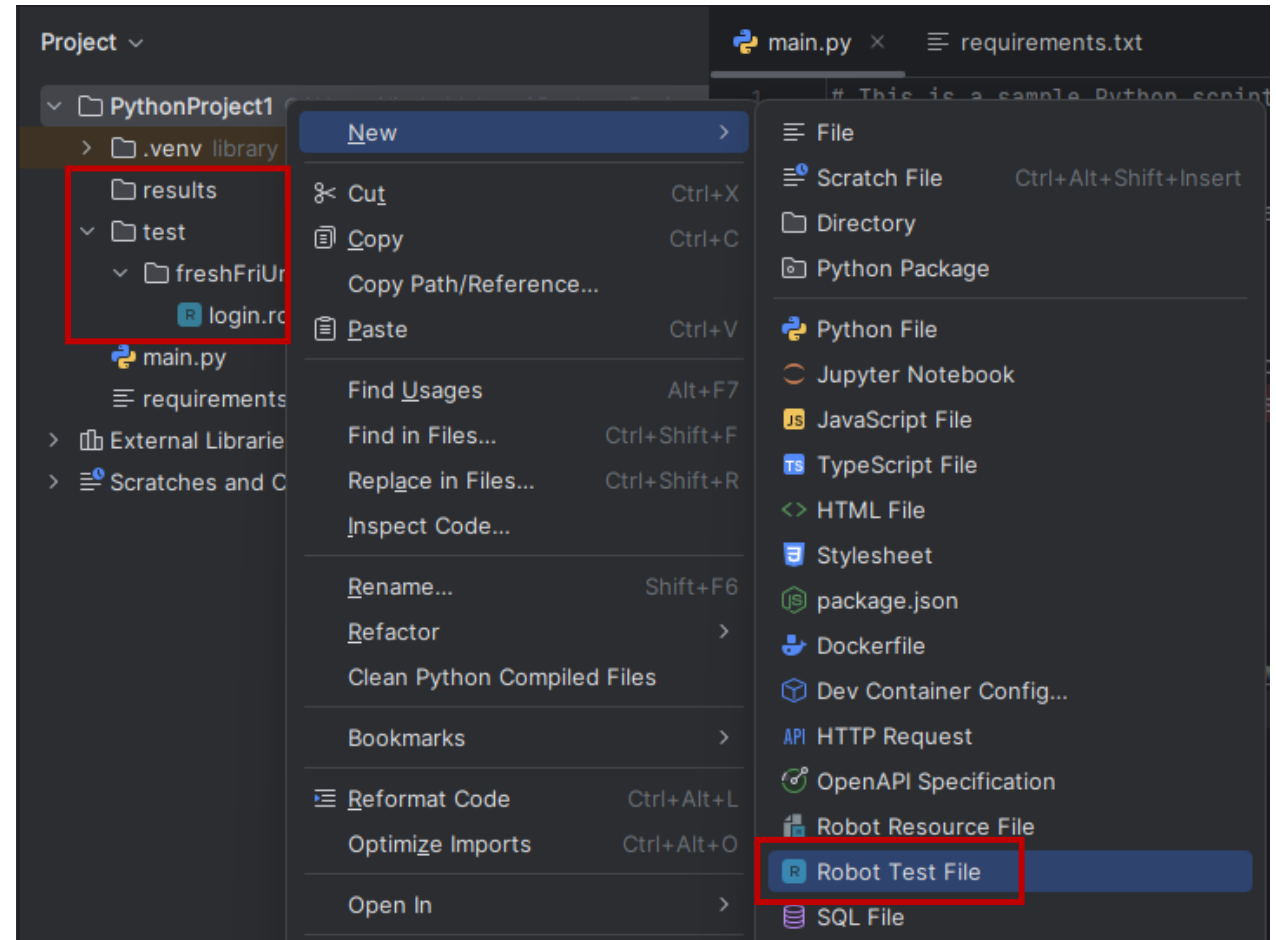
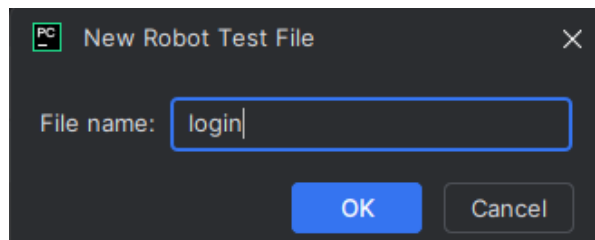
RobotFramework Plugin for PyCharm

- File → Settings → Plugins → Marketplace
- **RobotFramework Helper** plugin
RobotCode alternative for VS Code
- Support for writing and running scripts
Adding keywords, variables, tags, ..
Running tests
Debug
Documentation, ...



Preparing the project structure and creating the Robot Test File

- Adding folders (right click)
 - Results
 - Results of tests run
 - Test → FreshFriUniza
 - Scripts with tests
- Adding a Robot Test File
 - .robot script



Repository structure for Lab 5

Keep automated tests and reports organized

```
automation/  
└─ robot/  
   ├── README.md  
   ├── tests/  
   │   ├── login_tests.robot  
   │   └── project_tests.robot  
   ├── resources/  
   │   └── common_keywords.robot  
   └── results/  
       └── .gitkeep  
  
reports/  
└─ lab5-robot-framework-report.md
```

Why this matters

A clean structure makes the final project easier to evaluate. It also separates test logic, reusable keywords and generated results.

Do not commit unnecessary files

Generated logs can be large. Keep selected evidence, screenshots and the summary report.



Exercise 1: first Robot test

Show pass/fail and generated HTML outputs

```
*** Test Cases ***  
Simple Passing Test  
    Should Be Equal    Hello    Hello  
  
Simple Failing Test  
    Should Be Equal    Hello    World
```

```
robot automation/robot/tests/first_test.robot
```

Observe outputs

output.xml
log.html
report.html

Discussion

What information is visible in log.html?
How does a failed test look?
What evidence can be used in a final report?



Anatomy of a .robot file

Four sections you will use in this lab

Settings

- Libraries
- Resources
- Suite setup/teardown

Variables

- URLs
- Browser name
- Test data
- Credentials

Keywords

- Reusable actions
- Business steps
- Common checks

Test Cases

- Scenario names
- Steps
- Assertions
- Results

The goal is not only to run tests. The goal is to make tests readable and maintainable.



Libraries in Robot Framework

<https://robotframework.org/#resources>

<https://robotframework.org/SeleniumLibrary/SeleniumLibrary.html>

LIBRARIES

BUILT-IN

TOOLS

Separately developed external libraries that can be installed based on your needs. Creating your own libraries is a breeze. For instructions, see [creating test libraries](#) in Robot Framework User Guide.

Filter by tag

Name	Description	Stars	Tags
SeleniumLibrary	Web testing library that uses popular Selenium tool internally.	1304	WEB, SELENIUM
Browser Library	A modern web testing library powered by Playwright . Aiming for speed, reliability and visibility.	777	WEB
HTTP RequestsLibrary (Python)	HTTP level testing using Python Requests internally.	458	HTTP
AppiumLibrary	Android and iOS testing. Uses Appium internally.	371	MOBILE
REStinstance	Test library for HTTP JSON APIs.	195	HTTP
SSHLibrary	Enables executing commands on remote machines over an SSH connection. Also supports transferring files using SFTP.	143	FTP, SSH
Database Library (Python)	Python based library for database testing. Works with any Python interpreter, including Jython.	141	DB

SeleniumLibrary

Search

Keywords (182)

Add Cookie

Add Location Strategy

Alert Should Be Present

Alert Should Not Be Present

Assign Id To Element

Capture Element Screenshot

Capture Page Screenshot

Checkbox Should Be Selected

Checkbox Should Not Be Selected

Choose File

Clear Element Text

Click Button

Click Element

Click Element At Coordinates

Click Image

Click Link

Close All Browsers

Close Browser

Close Window

Cover Element

Create Webdriver

Current Frame Should Contain

Library version: 6.6.1

Library scope: GLOBAL

Introduction

SeleniumLibrary is a web testing library for Robot Framework.

This document explains how to use keywords provided by SeleniumLibrary. For information about installation, support, and more, please visit the [project pages](#). For more information about Robot Framework, see <http://robotframework.org>.

SeleniumLibrary uses the Selenium WebDriver modules internally to control a web browser. See <http://seleniumhq.org> for more information about Selenium in general and SeleniumLibrary README.rst [Browser drivers chapter](#) for more details about WebDriver binary installation.

- [Locating elements](#)
- [Browser and Window](#)
- [Browser and Driver options and service class](#)
- [Timeouts, waits, and delays](#)
- [Run-on-failure functionality](#)
- [Boolean arguments](#)
- [EventFiringWebDriver](#)
- [Thread support](#)
- [Plugins](#)
- [Language](#)
- [Importing](#)
- [Keywords](#)

Locating elements

All keywords in SeleniumLibrary that need to interact with an element on a web page take an argument typically named `locator` that specifies how to find the element. Most often the locator is given as a string using the locator syntax described below, but [using WebElements](#) is possible too.

Locator syntax

SeleniumLibrary supports finding elements based on different strategies such as the element id, XPath expressions, or CSS selectors. The



Using libraries = documentation

- Library documentation
 - Defined keywords
 - Required arguments
 - Usage examples

Click Element

Click Element At Coordinates

Click Image

Click Link

Close All Browsers

Close Browser

Close Window

Click Element

Arguments

locator		WebElement str
modifier	= False	bool str
action_chain	= False	bool

Documentation

Click the element identified by `locator`.

See the [Locating elements](#) section for details about the locator syntax.

The `modifier` argument can be used to pass [Selenium Keys](#) when clicking the element. The `+` can be used as a separator for different Selenium Keys. The `CTRL` is internally translated to the `CONTROL` key. The `modifier` is space and case insensitive, example "alt" and "alt " are supported formats to [ALT key](#). If `modifier` does not match to Selenium Keys, keyword fails.

If `action_chain` argument is true, see [Boolean arguments](#) for more details on how to set boolean argument, then keyword uses `ActionChain` based click instead of the `<web_element>.click()` function. If both `action_chain` and `modifier` are defined, the click will be performed using `modifier` and `action_chain` will be ignored.

Example:

Click Element	id:button		# Would click element without any modifiers.
Click Element	id:button	CTRL	# Would click element with CTRL key pressed down.
Click Element	id:button	CTRL+ALT	# Would click element with CTRL and ALT keys pressed down.
Click Element	id:button	action_chain=True	# Clicks the button using an Selenium ActionChains

The `modifier` argument is new in SeleniumLibrary 3.2 The `action_chain` argument is new in SeleniumLibrary 4.1



Exercise 2: web test with SeleniumLibrary

Valid login scenario

```
*** Settings ***
Library      SeleniumLibrary

*** Variables ***
${URL}       https://www.saucedemo.com/
${BROWSER}   Chrome
${USERNAME}  standard_user
${PASSWORD}  secret_sauce

*** Test Cases ***
Login With Valid Credentials
    Open Browser    ${URL}    ${BROWSER}
    Input Text      id:user-name    ${USERNAME}
    Input Text      id:password    ${PASSWORD}
    Click Button    id:login-button
    Page Should Contain    Products
    Close Browser
```

Key idea

Use SeleniumLibrary to automate the browser, but write the test as readable steps.

Assertion

Page Should Contain
Products

Run command

robot automation/robot/tests/login_tests.robot

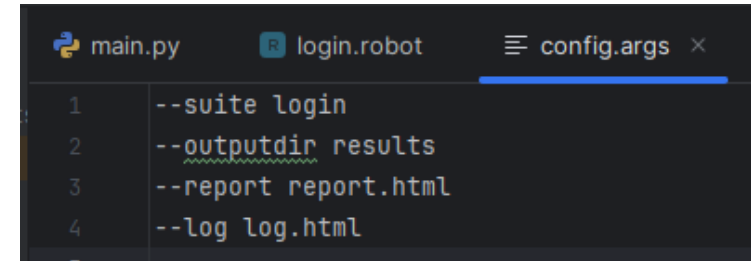


Running Tests – Argument Definition

- Creating a conf.args file with switches
 - <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#argument-files>

Conf.args

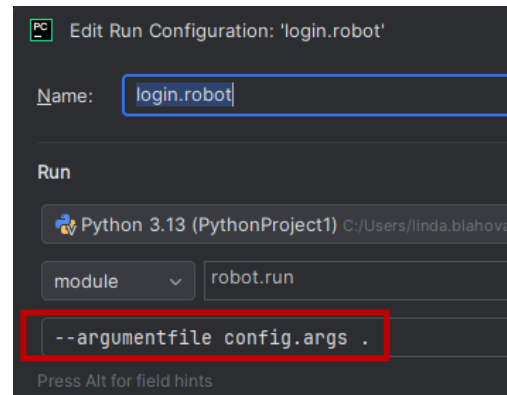
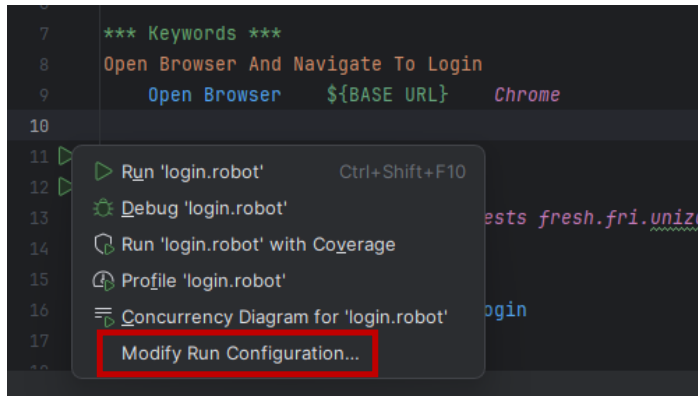
```
-- suite login
-- outputdir results
-- report report.html
-- log log.html
```



The screenshot shows an IDE with three tabs: main.py, login.robot, and config.args. The config.args tab is active and displays the following content:

```
1 --suite login
2 --outputdir results
3 --report report.html
4 --log log.html
```

- Using a file as argumentFile for a robot in the configuration for running tests



Conf.args

```
-- argumentfile config.args .
```

- Alternative - run with the command: `python.exe -m robot.run --argumentfile config.args .`



Generated report

- report.html → right click → open in → browser
- Results statistics
- Tags, Suites

PythonProject1 Report

Generated
20241201 16:11:50 UTC+01:00
51 minutes 42 seconds ago

Summary Information

Status:

All tests passed

Start Time:

20241201 16:11:43.140

End Time:

20241201 16:11:50.391

Elapsed Time:

00:00:07.251

Log File:

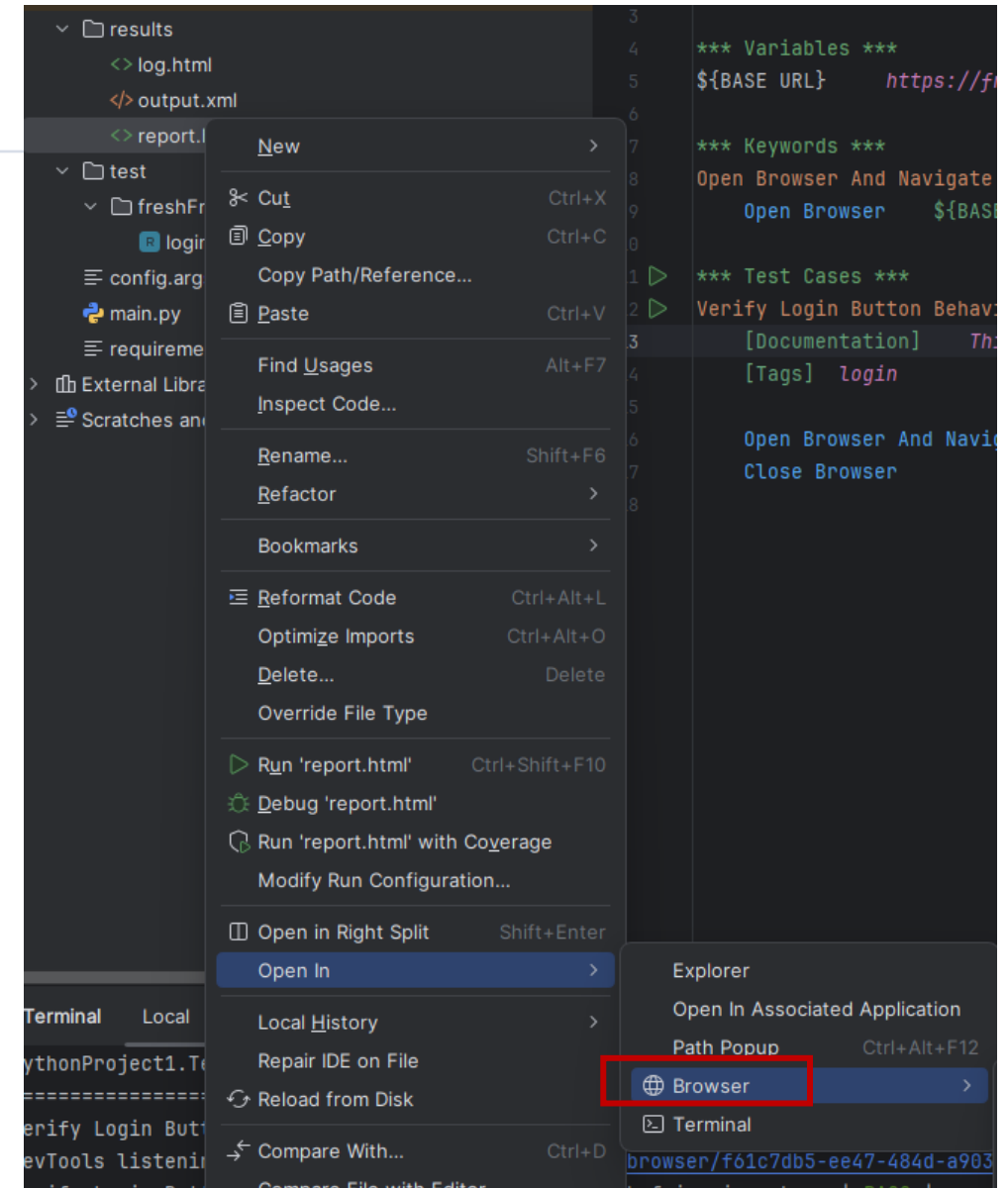
log.html

Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	1	1	0	0	00:00:07	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
login	1	1	0	0	00:00:07	

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
PythonProject1	1	1	0	0	00:00:07	
PythonProject1 . Test	1	1	0	0	00:00:07	
PythonProject1 . Test . freshFriUniza	1	1	0	0	00:00:07	
PythonProject1 . Test . freshFriUniza . Login	1	1	0	0	00:00:07	



Exercise 3: negative login test

Automated tests should also verify validation and error handling

*** Test Cases ***

Login With Invalid Password

Open Browser \${URL} \${BROWSER}

Input Text id:user-name standard_user

Input Text id:password wrong_password

Click Button id:login-button

Page Should Contain Username and password do not
match

Close Browser

Why negative tests?

They verify that the application handles invalid inputs safely and clearly.

Expected result

An error message is visible and the user is not logged in.



Variables

Remove hard-coded values from test cases

```
*** Variables ***
${URL}             https://www.saucedemo.com/
${BROWSER}         Chrome
${USERNAME}         standard_user
${PASSWORD}         secret_sauce
${WRONG_PASSWORD}   wrong_password
```

- Change test data in one place
- Reuse the same URL/browser across tests
- Make the test suite easier to configure
- Prepare for multiple environments later

Rule: if a value appears repeatedly, it is a candidate for a variable.



Exercise 4: create custom keywords

Turn repeated actions into reusable steps

*** Keywords ***

Open Tested Application

[Arguments]	\${url}	\${browser}
Open Browser	\${url}	\${browser}

Login To Application

[Arguments]	\${username}	\${password}
Input Text	id:user-name	\${username}
Input Text	id:password	\${password}
Click Button	id:login-button	

Verify Products Page

Page Should Contain	Products
---------------------	----------

Close Tested Application

Close Browser

Keyword design

- Use names that describe business actions, not only technical clicks.

Examples

- Login To Application
- Verify Products Page
- Add Product To Cart
- Search For Product



Resource files

Separate reusable keywords from test cases

```
*** Settings ***
Resource    ../resources/common_keywords.robot

*** Test Cases ***
Login With Valid Credentials Using Keywords
    Open Tested Application    ${URL}    ${BROWSER}
    Login To Application       ${USERNAME}    ${PASSWORD}
    Verify Products Page
    Close Tested Application
```

Benefit

Test cases become shorter and easier to read.

Project rule

Put common reusable keywords into
automation/robot/resources/common_keywords.robot



Example: add product to cart

A reusable e-shop regression test

```
*** Test Cases ***  
Add Product To Cart  
  Open Tested Application    ${URL}    ${BROWSER}  
  Login To Application       ${USERNAME}  ${PASSWORD}  
  Click Button               id:add-to-cart-sauce-labs-backpack  
  Page Should Contain Element    class:shopping_cart_badge  
  Close Tested Application
```

Manual origin

TC-006 Add product to cart

Assertion

Cart badge is visible after adding a product.

Automation value

Fast regression check after UI changes.



Team exercise

Rewrite your own scenario from Lab 4

Minimum requirement

- 2 Robot Framework test cases
- 1 positive test
- 1 negative or validation test
- variables used
- at least 2 custom keywords
- HTML report generated

Recommended scenarios

- Login
- Search
- Add to cart
- Form validation
- Navigation
- Checkout without payment

Start with a scenario that already worked in Selenium.



What to document

Make the result understandable for the final defense

automation/robot/README.md

- tested application
- how to install
- how to run tests
- known limitations

reports/lab5-robot-framework-report.md

- automated test cases
- keywords created
- execution result
- problems and notes

Evidence

- report.html screenshot
- log.html screenshot
- terminal output
- repository link



Lab 5 report template

Use this structure in your repository

```
# Lab 5 - Robot Framework Report

## 1. Tested application
Name:
URL:

## 2. Automated test cases
| Test Case ID | Title | Type | Result |
|---|---|---|---|
| TC-001 | Login with valid credentials | Positive | Passed |

## 3. Keywords created
| Keyword | Purpose |
|---|---|
| Login To Application | Performs login |

## 4. Test execution
Command used:
robot automation/robot/tests/project_tests.robot

## 5. Problems and notes
```



Short comparison task

Add 5-7 sentences to your report

Answer these questions

- What was easier in Robot Framework?
- What was more difficult?
- Would you use Robot Framework for this project?
- Why?

Example answer

Robot Framework was easier to read because test steps were written as keywords. Reusing login steps was simpler. Debugging locators was still necessary. For this project, Robot is suitable for regression tests such as login, search and cart testing.



Lab 5 checkpoint

Submit by the end of the lab 6

Repository URL:
Number of Robot tests:
Positive test: yes/no
Negative or validation test: yes/no
Custom keywords: yes/no
HTML report generated: yes/no
Main problem:

Minimum output

- 2 Robot tests
- variables used
- 2 custom keywords
- HTML report/log
- Lab 5 markdown report
- Git push



Lab 5 evaluation

Suggested 15-point checkpoint

Criterion	Points
Functional Robot Framework installation and execution	2
At least 2 Robot test cases	5
Use of variables	1
Use of custom keywords	2
HTML report/log generated	2
Repository documentation	1
Selenium vs Robot comparison	2
Total	15



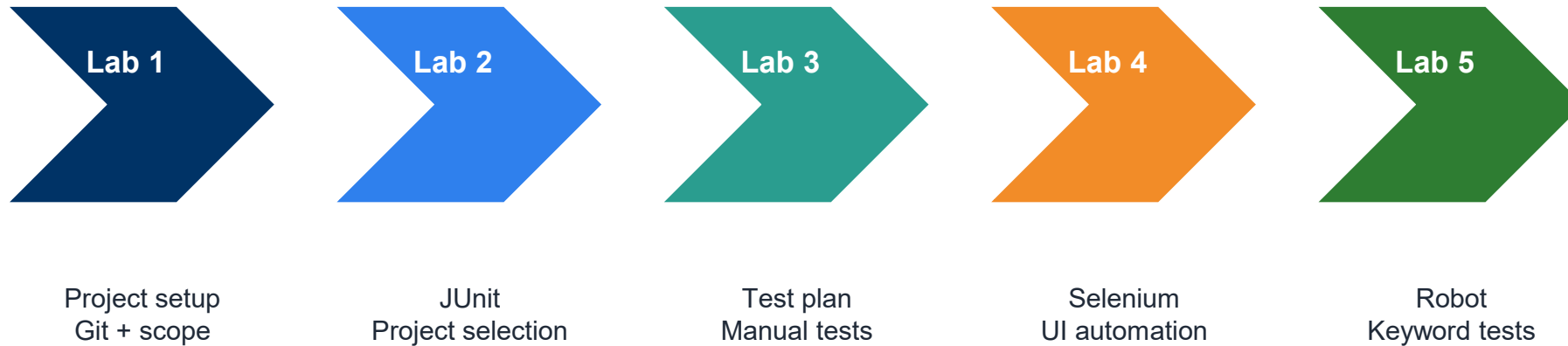


Thank you for your attention.

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk

From labs to final project

What each lab contributes to the final submission



The defense should show the complete testing story, not just separate lab outputs.



Goal of the final project

Demonstrate a complete basic testing process

Plan

Define test scope, objectives, risks, environment and exit criteria.

Design

Create manual test cases using selected test design techniques.

Execute

Run manual, UI automation, keyword-driven and API tests.

Report

Summarize results, defects, problems and limitations.

Defend

Present the work and explain the contribution of each team member.



Final submission package

Everything must be available before the defense

- Complete Git repository with clear folder structure
- Final report in PDF or Markdown format
- Final presentation in PPTX or PDF format
- Manual test cases and at least one defect report
- Selenium, Robot Framework and Postman outputs
- Summary of test execution results

Submission deadline

Recommended: 9 July 2026, 23:59

On 10 July 2026 there should be only defense and evaluation, not project completion.



Recommended repository structure

Make the project easy to inspect and evaluate

```
team-project/  
├── README.md  
├── docs/  
│   ├── team.md  
│   ├── project-selection.md  
│   ├── project-charter.md  
│   └── test-plan.md  
├── manual-tests/  
│   ├── test-case-template.md  
│   └── manual-test-cases.md  
├── automation/  
│   ├── selenium/  
│   └── robot/  
├── reports/  
└── final/
```

Do not submit a chaotic ZIP file.

Use meaningful file names.

Keep screenshots and exported reports in reports/.

Add short README files for automation folders.

Make sure the repository is accessible to the teacher.



Final report requirements

Recommended length: 8–15 pages

- | | | | |
|---|--------------------|----|-----------------|
| 1 | Introduction | 9 | Selenium tests |
| 2 | Team and roles | 10 | Robot tests |
| 3 | Tested application | 11 | Results summary |
| 4 | Test scope | 12 | Limitations |
| 5 | Test plan | 13 | Lessons learned |
| 6 | Manual test cases | 14 | Conclusion |
| 7 | Defect report | | |
| 8 | JUnit exercise | | |



Final presentation

Recommended: 8–10 slides

1 Team + application

2 Application scope

3 Test strategy

4 Manual test cases

5 Defect report

6 Selenium automation

7 Robot automation

8 Postman API - exercise

9 Results summary

10 Lessons learned



Evaluation rubric

Final defense: 45 points

Criterion	Points
Tested application and scope	5
Test plan and manual test cases	8
Selenium + Robot Framework automation	8
Postman API testing	5
Final report and repository structure	5
Results, defects and interpretation	4
Answers of team members	5

+5 point
presentation



Final message

Testing is not only execution — it is communication of evidence

A good final project tells a complete story:

What did we test?

How did we design
tests?

What did we
automate?

What did we find?

What did we learn?

Be clear. Be honest. Show evidence.

